

Elliot Lebreton / BTS SIO

DÉPLOIEMENT AUTOMATIQUE DE MACHINES VIRTUELLES AVEC

PACKER, CLOUD-INIT ET TERRAFORM

Sommaire

Objectif.....	3
Prérequis.....	3
1. Création de l'utilisateur Packer.....	3
2. Attribution du rôle PVEAdmin.....	3
3. Génération du token API.....	3
4. Configuration du service DHCP.....	3
5. Vérification de la version Proxmox.....	4
6. Installation de Packer et Terraform.....	5
Configuration de Packer.....	6
Structure du projet.....	6
Description des fichiers.....	6
Initialisation et exécution.....	9
Résultat de l'exécution.....	9
Explication des étapes.....	10
Configuration de Terraform.....	11
Structure du projet.....	11
Description des fichiers.....	11
Initialisation et exécution.....	13
Notes importantes.....	16

Objectif

Industrialiser la création et le déploiement de machines virtuelles Ubuntu à l'aide de Packer et Terraform.

Prérequis

1. Création de l'utilisateur Packer

Tout d'abord, nous allons créer un utilisateur **packer@pve** dans Proxmox VE avec le mot de passe **packer1234!**

```
pveum user add packer@pve --password packer1234!
```

2. Attribution du rôle PVEAdmin

On donne à l'utilisateur **packer@pve** le rôle **PVEAdmin** à la racine de l'infrastructure. PVEAdmin est un rôle très puissant (presque tous les droits) et permet à l'utilisateur de cloner des templates, créer des VMs, modifier des ressources, etc.

```
pveum aclmod / -user packer@pve -role PVEAdmin
```

3. Génération du token API

Nous allons maintenant générer un token API pour **packer@pve** et l'appeler packer. Ce token sera utilisé par Packer et Terraform pour s'authentifier à l'API Proxmox sans mot de passe.

```
## -expire 0 : le token n'expire jamais  
## -privsep 0 : le token hérite des permissions de l'utilisateur packer@pve  
pveum user token add packer@pve packer -expire 0 -privsep 0
```

4. Configuration du service DHCP

Ensuite, il est vivement recommandé d'avoir un **service DHCP** valide et fonctionnel afin que le déploiement soit plus simple et d'éviter de mettre une IP statique à chaque fois. Nous pouvons par exemple utiliser **isc-dhcp-server** sur une machine Linux.

5. Vérification de la version Proxmox

Le serveur Proxmox-SP2 est sous une ancienne version de Proxmox (v7.3). Il faut donc une version adaptée pour Packer et Terraform. Pour voir la version actuelle de Proxmox, il suffit de faire cette commande :

```
pveversion
```

6. Installation de Packer et Terraform

Pour cette version de Proxmox, nous allons utiliser la version **1.1.0** du plugin **HashiCorp** afin d'utiliser Packer.

```
# 1. Installation des dépendances de base
apt install -y wget unzip gnupg software-properties-common

# 2. Aller dans le dossier /tmp
cd /tmp

# 3. Télécharger la version 1.1.0 de hashicorp pour packer
wget
https://releases.hashicorp.com/packer/1.12.0/packer_1.12.0_linux_
_amd64.zip

# 4. Décompresser le dossier zip téléchargé
unzip packer_1.12.0_linux_amd64.zip

# 5. Déplacer dans un dossier accessible globalement
sudo mv packer /usr/local/bin/

# 6. Vérifier la version de packer
packer -version

-----

# 7. Télécharger la version 1.6.6 de hashicorp pour terraform
wget
https://releases.hashicorp.com/terraform/1.6.6/terraform_1.6.6_1
inux_amd64.zip

# 8. Décompresser le dossier zip téléchargé
unzip terraform_1.6.6_linux_amd64.zip
```

```
# 9. Déplacer dans un dossier accessible globalement
```

```
sudo mv terraform /usr/local/bin/
```

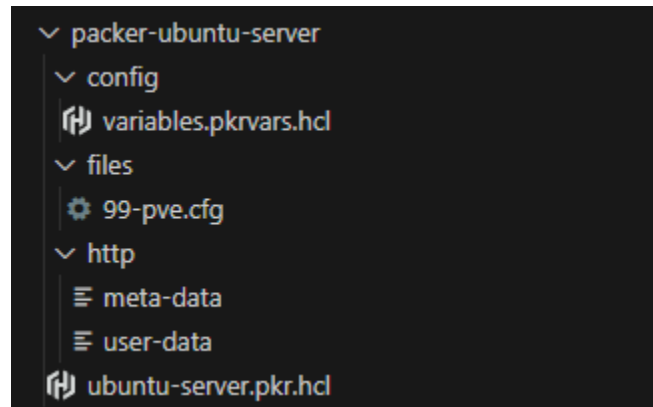
```
# 10. Vérifier la version de terraform
```

```
terraform -version
```

Configuration de Packer

Structure du projet

Nous allons créer un dossier **packer-ubuntu-server** où nous allons mettre nos fichiers. Voici l'arborescence :



Description des fichiers

packer-ubuntu-server/

C'est le dossier racine de notre projet Packer.

config/

Contient les fichiers de configuration avec des variables, typiquement utilisées pour éviter de modifier le fichier principal.

variables.pkrvars.hcl

Fichier HCL qui contient les différentes variables. Exemple ici :

```
## GENERAL SETTINGS
api_url           = "https://localhost:8006/api2/json/"
api_token_id      = "packer@pve!packer"
api_token_secret  = "85d69d2f-75a6-4a4d-a54d-dbb12863c67d"
node_name         = "ProxMox-SP1-SIO"
template_vm_id    = "9999"
template_vm_name  = "packer-ubuntu-server"
template_vm_description = "Ubuntu Server noble image"

## VM SETTINGS
iso_file          = "local:iso/ubuntu-24.04.4-live-server-amd64.iso"
iso_storage_pool  = "local"
unmount_iso       = true
qemu_agent        = true
scsi_controller  = "virtio-scsi-single"
cores             = "4"
memory           = "4096"
```

files/

Dossier pour stocker les fichiers à injecter dans la VM.

99-pve.cfg

Fichier utilisé pour configurer le réseau ou d'autres options.

[http/](#)Ce dossier est servi en HTTP par Packer pendant l'installation.

meta-data

Contient le nom d'hôte et l'UUID de la VM qui sera créée.

user-data

Contient le script de préconfiguration pour la VM (utilisateur, mot de passe, SSH, packages...). Exemple ici :

```
## cloud-config
autoinstall:
  version: 1
  locale: fr_FR
  keyboard:
    layout: fr
  ssh:
    install-server: true
    allow-pw: true
    disable_root: true
    ssh_quiet_keygen: true
    allow_public_ssh_keys: true
  packages:
    - qemu-guest-agent
    - sudo
    - htop
    - nload
    - wget
    - curl
```

ubuntu-server.pkr.hcl

Le fichier principal Packer. Il définit les variables, la source, le build et les provisioners.

Initialisation et exécution

10. Vérifier la version de terraform

```
terraform -version
```

Enfin, pour pouvoir lancer la création d'une VM avec Packer et créer une template prête à l'emploi avec Cloud-Init, il suffit de lancer cette commande :

```
# Lancer ubuntu-server.pkr.hcl avec le fichier des variables
packer build -on-error=abort
-var-file=config/variables.pkrvars.hcl ubuntu-server.pkr.hcl
```

Résultat de l'exécution

Voici le résultat obtenu après le lancement du fichier `ubuntu-server.pkr.hcl` :

```
root@DC01-D02-22-x9:/home/packer-ubuntu-server# packer build -on-error=abort -var-file=config/variables.pkrvars.hcl ubuntu-server.pkr.hcl
ubuntu-server.proxmox-iso.ubuntu-server: output will be in this color.

==> ubuntu-server.proxmox-iso.ubuntu-server: Creating VM
==> ubuntu-server.proxmox-iso.ubuntu-server: No VM ID given, getting next free from Proxmox
==> ubuntu-server.proxmox-iso.ubuntu-server: Starting VM
==> ubuntu-server.proxmox-iso.ubuntu-server: Starting HTTP server on port 8393
==> ubuntu-server.proxmox-iso.ubuntu-server: Waiting 10s for boot
==> ubuntu-server.proxmox-iso.ubuntu-server: Typing the boot command
==> ubuntu-server.proxmox-iso.ubuntu-server: Waiting for SSH to become available...
==> ubuntu-server.proxmox-iso.ubuntu-server: Connected to SSH!
==> ubuntu-server.proxmox-iso.ubuntu-server: Provisioning with shell script: /tmp/packer-shell11799930274
ubuntu-server.proxmox-iso.ubuntu-server: Waiting for cloud-init...
ubuntu-server.proxmox-iso.ubuntu-server: Waiting for cloud-init...
ubuntu-server.proxmox-iso.ubuntu-server: Waiting for cloud-init...

ubuntu-server.proxmox-iso.ubuntu-server: Reading package lists...
ubuntu-server.proxmox-iso.ubuntu-server: Building dependency tree...
ubuntu-server.proxmox-iso.ubuntu-server: Reading state information...
ubuntu-server.proxmox-iso.ubuntu-server: 0 upgraded, 0 newly installed, 0 to remove and 1 not upgraded.
ubuntu-server.proxmox-iso.ubuntu-server: Reading package lists...
ubuntu-server.proxmox-iso.ubuntu-server: Building dependency tree...
ubuntu-server.proxmox-iso.ubuntu-server: Reading state information...
==> ubuntu-server.proxmox-iso.ubuntu-server: Uploading files/99-pve.cfg => /tmp/99-pve.cfg
ubuntu-server.proxmox-iso.ubuntu-server: 99-pve.cfg 39 B / 39 B [=====]
==> ubuntu-server.proxmox-iso.ubuntu-server: Provisioning with shell script: /tmp/packer-shell12259010866
==> ubuntu-server.proxmox-iso.ubuntu-server: Stopping VM
==> ubuntu-server.proxmox-iso.ubuntu-server: Converting VM to template
==> ubuntu-server.proxmox-iso.ubuntu-server: Adding a cloud-init cdrom in storage pool local-lvm
Build 'ubuntu-server.proxmox-iso.ubuntu-server' finished after 10 minutes 50 seconds.

==> Wait completed after 10 minutes 50 seconds

==> Builds finished. The artifacts of successful builds are:
--> ubuntu-server.proxmox-iso.ubuntu-server: A template was created: 100
```

Explication des étapes

Creating VM, No VM ID given, Starting VM signifie que le script va créer et lancer une VM avec le premier ID disponible car aucun n'a été défini au préalable.

Starting HTTP server on port 8393, Waiting 10s for boot, Typing the boot command signifie que Packer ouvre un serveur HTTP local sur le port 8393 pour y injecter les fichiers nécessaires via Cloud-Init en simulant une frappe de clavier pour lancer l'installation automatiquement.

Waiting for SSH to become available, Connected to SSH signifie que Packer attend que le serveur SSH de la VM soit prêt.

Provisioning with shell script: /tmp/packer-shell1799930274 signifie que le script shell **Provisioning #1** (présent dans le fichier pkr.hcl) est exécuté dans la VM afin de nettoyer et préparer la template.

Waiting for cloud-init signifie que Packer attend que cloud-init termine sa configuration initiale dans la VM.

Reading package lists, Building dependency tree, Reading state information, 0 upgraded, 0 newly installed... signifie que cloud-init met à jour les paquets automatiquement (il est indiqué dans le fichier user-data) :

```
package_upgrade: true
```

Uploading files/99-pve.cfg => /tmp/99-pve.cfg correspond à l'exécution du script shell **Provisioning #2**.

Provisioning with shell script: /tmp/packer-shell12259010866 correspond à l'exécution du script shell **Provisioning #3**.

Stopping VM, Converting VM to template, Adding a cloud-init cdrom montre que Packer a finalisé la création de la VM. Il va donc l'arrêter, la convertir en template et ajouter le disque cloud-init sur le stockage local-lvm.

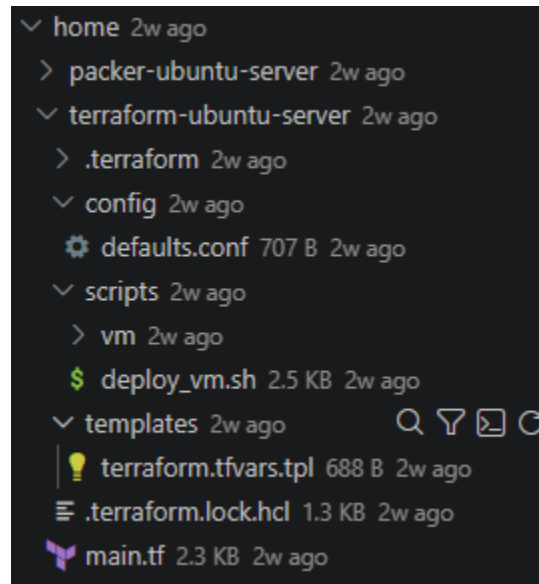
	Memory	4.00 GiB
	Processors	4 (1 sockets, 4 cores) [kvm64]
	BIOS	Default (SeaBIOS)
	Display	Default
	Machine	Default (i440fx)
	SCSI Controller	VirtIO SCSI single
	CloudInit Drive (ide0)	local-lvm:vm-104-cloudinit,media=cdrom
	Hard Disk (scsi0)	local-lvm:base-104-disk-0,cache=none,replicate=0,size=10G
	Network Device (net0)	virtio=1E:F9:BC:F0:36:0C,bridge=vmbr0

Cette template pourra ensuite nous servir de « base » afin de déployer automatiquement des VMs prêtes à l'emploi rapidement avec Terraform que nous allons voir ci-dessous.

Configuration de Terraform

Structure du projet

Nous allons créer un dossier **terraform-ubuntu-server** où nous allons mettre nos fichiers. Voici l'arborescence :



Description des fichiers

terraform-ubuntu-server/

C'est le dossier racine de notre projet Terraform.

config/defaults.conf

Fichier qui contient les différentes variables par défaut qui vont servir à personnaliser notre machine pour l'utiliser directement.

scripts/vm/

Dossier contenant tous les réglages des différentes VMs qui ont été créées avec Terraform.

scripts/deploy_vm.sh

Script intuitif afin de personnaliser la mise en place de VMs en personnalisant quelques options, comme par exemple le nombre de cœurs, la RAM ou le réseau.

templates/terraform.tfvars.tpl

Ce fichier contient toutes les variables du fichier **defaults.conf** afin de les interpréter sur le fichier **main.tf**.

```
api_url          = "${api_url}"
api_token_id     = "${api_token_id}"
api_token_secret = "${api_token_secret}"
target_node      = "${target_node}"
template_name    = "${template_name}"
vm_name          = "${vm_name}"
vm_id            = ${vm_id}
cores            = ${cores}
memory           = ${memory}
network_bridge   = "${network_bridge}"
ci_user          = "${ci_user}"
ci_password      = "${ci_password}"
ipconfig0        = "${ipconfig0}"
```

main.tf

Fichier d'exécution pour créer rapidement des VMs selon nos besoins avec les variables préalablement définies. Ce fichier est lancé grâce au script **deploy_vm.sh** afin de poser des questions à l'utilisateur pour personnaliser la VM à sa guise.

Initialisation et exécution

Commande à faire avant de lancer terraform pour initialiser le répertoire et installer les plugins nécessaires

Terraform init

```
root@DC01-D02-22-x9:/home/terraform-ubuntu-server# terraform init

Initializing the backend...

Initializing provider plugins...
- Finding telmate/proxmox versions matching "2.9.11"...
- Installing telmate/proxmox v2.9.11...
- Installed telmate/proxmox v2.9.11 (self-signed, key ID A9EBBE091B35AFCE)

Partner and community providers are signed by their developers.
If you'd like to know more about provider signing, you can read about it here:
https://www.terraform.io/docs/cli/plugins/signing.html

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!
```

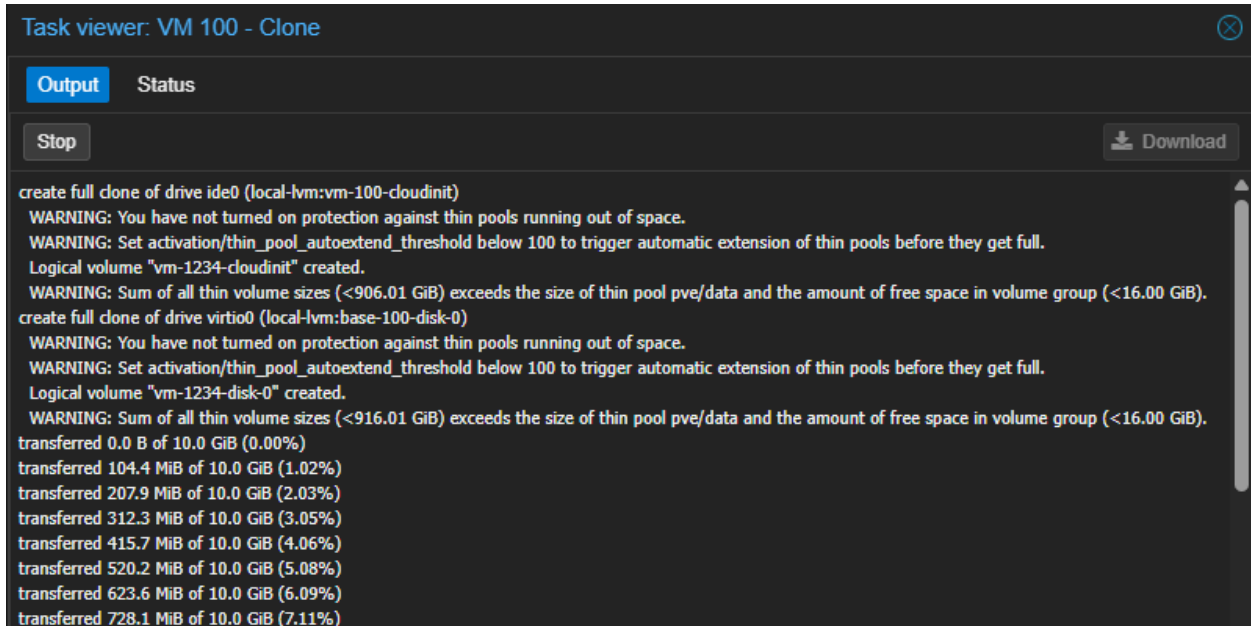
Enfin, pour pouvoir lancer la création d'une VM avec Terraform et créer une VM prête à être utilisée, il suffit de lancer cette commande :

Lance le script deploy_vm.sh (qui exécute main.tf) pour déployer simplement une VM avec des options personnalisées
./scripts/deploy_vm.sh

Grâce à ce script, vous pouvez personnaliser certaines options de notre VM qui sera déployée, par exemple le nombre de cœurs ou le mot de passe.

```
root@DC01-D02-22-x9:/home/terraform-ubuntu-server# ./scripts/deploy_vm.sh
Nom de la VM [VM-DEFAULT] : TEST-TERRAFORM
ID de la VM [3000] : 1234
Nombre de coeurs CPU [2] : 4
Mémoire RAM en Mo [2048] : 8192
Bridge réseau [vmbr0] :
Bridge réseau (ex ip static: ip=192.168.1.100/24,gw=192.168.1.1) [ip=dhcp] :
Nom d'utilisateur SSH [ubuntu] : terraform
Mot de passe SSH [ubuntu] : Password1234
```

Ensuite, on peut voir que la template **packer-ubuntu-server** (id 100) qui a été définie dans le fichier defaults.conf est en train d'être clonée pour être utilisée.



```
Task viewer: VM 100 - Clone

Output Status

Stop Download

create full clone of drive ide0 (local-lvm:vm-100-cloudinit)
WARNING: You have not turned on protection against thin pools running out of space.
WARNING: Set activation/thin_pool_autoextend_threshold below 100 to trigger automatic extension of thin pools before they get full.
Logical volume "vm-1234-cloudinit" created.
WARNING: Sum of all thin volume sizes (<906.01 GiB) exceeds the size of thin pool pve/data and the amount of free space in volume group (<16.00 GiB).
create full clone of drive virtio0 (local-lvm:base-100-disk-0)
WARNING: You have not turned on protection against thin pools running out of space.
WARNING: Set activation/thin_pool_autoextend_threshold below 100 to trigger automatic extension of thin pools before they get full.
Logical volume "vm-1234-disk-0" created.
WARNING: Sum of all thin volume sizes (<916.01 GiB) exceeds the size of thin pool pve/data and the amount of free space in volume group (<16.00 GiB).
transferred 0.0 B of 10.0 GiB (0.00%)
transferred 104.4 MiB of 10.0 GiB (1.02%)
transferred 207.9 MiB of 10.0 GiB (2.03%)
transferred 312.3 MiB of 10.0 GiB (3.05%)
transferred 415.7 MiB of 10.0 GiB (4.06%)
transferred 520.2 MiB of 10.0 GiB (5.08%)
transferred 623.6 MiB of 10.0 GiB (6.09%)
transferred 728.1 MiB of 10.0 GiB (7.11%)
```

```
## TEMPLATE SETTINGS
target_node="Proxmox-SP1-SIO"
template_name="packer-ubuntu-server"
```

Sur notre console SSH, on remarque que Terraform est en train de créer la VM en tenant compte des options que nous avons renseignées sur le script **deploy_vm.sh** (exemple : cores, memory, name...).

```
Terraform will perform the following actions:

# proxmox_vm_qemu.vm will be created
+ resource "proxmox_vm_qemu" "vm" {
  + additional_wait      = 5
  + agent                = 1
  + automatic_reboot     = true
  + balloon              = 0
  + bios                 = "seabios"
  + boot                 = (known after apply)
  + bootdisk             = (known after apply)
  + cipassword           = (sensitive value)
  + ciuser               = "terraform"
  + clone                = "packer-ubuntu-server"
  + clone_wait           = 10
  + cores                = 4
  + cpu                  = "host"
  + default_ipv4_address = (known after apply)
  + define_connection_info = true
  + force_create         = false
  + full_clone           = true
  + guest_agent_ready_timeout = 100
  + hotplug              = "network,disk,usb"
  + id                   = (known after apply)
  + ipconfig0            = "ip=dhcp"
  + kvm                  = true
  + memory               = 8192
  + name                 = "TEST-TERRAFORM"
```

Après environ 3 minutes, la VM a été créée avec succès. C'est effectivement beaucoup moins long que le faire à la main à chaque fois.

De plus, comme on l'a vu avec Packer, on a la possibilité d'installer des paquets de base, comme par exemple **htop** ou **unzip**.

```
proxmox_vm_qemu.vm: Still creating... [2m30s elapsed]
proxmox_vm_qemu.vm: Still creating... [2m40s elapsed]
proxmox_vm_qemu.vm: Still creating... [2m50s elapsed]
proxmox_vm_qemu.vm: Creation complete after 2m57s [id=DC01-D02-22-x9/qemu/1234]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
root@DC01-D02-22-x9:/home/terraform-ubuntu-server#
```

```
terraform@TEST-TERRAFORM:~$ sudo apt update
Atteint :1 http://security.ubuntu.com/ubuntu noble-security InRelease
Atteint :2 http://co.archive.ubuntu.com/ubuntu noble InRelease
Atteint :3 http://co.archive.ubuntu.com/ubuntu noble-updates InRelease
Atteint :4 http://co.archive.ubuntu.com/ubuntu noble-backports InRelease
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
1 paquet peut être mis à jour. Exécutez « apt list --upgradable » pour le voir.
terraform@TEST-TERRAFORM:~$
```

En allant sur le panel Proxmox, on remarque que Cloud-Init a bien pris en compte notre **utilisateur** et **notre mot de passe**, et nous pouvons nous connecter à notre VM et l'utiliser comme bon nous semble.

User	ubuntu
Password	*****
DNS domain	use host settings
DNS servers	use host settings
SSH public key	none
Upgrade packages	Yes
IP Config (net0)	ip=dhcp

Notes importantes

Note 1 : Si la version du serveur Proxmox est supérieure à la **version 8.1**, le provider Terraform nommé **telmate/proxmox, version 2.9.11** contient des bugs au niveau de l'interprétation des nombres. Pour cela, il suffit de remplacer ce provider par **thegameprofi/proxmox, version 2.9.15**.

```
## Déclaration du provider Terraform
terraform {
  required_providers {
    proxmox = {
      source = "thegameprofi/proxmox"
      version = "2.9.15"
    }
  }
}
```

Note 2 : Il peut y avoir des problèmes d'exécution du script s'il n'a pas les permissions requises. Pour cela, il vous suffit d'ajouter les droits d'exécution du script avec cette commande :

```
# +x : ajoute le droit d'exécution à deploy_vm.sh
chmod +x ./scripts/deploy_vm.sh
```

Note 3 : Il peut aussi y avoir quelques problèmes d'interprétations du fichier **defaults.conf**. Pour cela, convertir le fichier au format **Unix** à la place du format **DOS** (Windows) peut régler le problème :

```
dos2unix ./config/defaults.conf
```